仓库链接：https://github.com/zechun115/qhf（分支：大作业）

姓名：邱浩峰　　提交日期：2026 年 10 月 29 日

课程大作业 · 第二阶段（需求变更迭代）

大作业—第二阶段（需求变更）项目报告

基于 MongoDB 的在线问卷系统 · 题目库与题目版本管理

项目名称	基于 MongoDB 的在线问卷系统
技术栈	Node.js • Express • Mongoose • MongoDB
交付内容	需求变更分析 • 数据结构演进 • 可运行实现 • 自动化测试
测试规模	88 个自动化用例全部通过（第二阶段新增 19 条）
运行方式	npm install → npm run seed → npm start（测试：npm test）

2026 年 10 月

一、需求变更概述

系统在第一阶段交付并"上线使用"后，收到用户反馈共八项需求变更，全部围绕一个主题：**题目应当脱离问卷独立管理**。原始需求整理如下：

编号	用户反馈	期望效果
1	每次做类似问卷都要重写题目，太麻烦	可创建并保存常用题；建新问卷时可直接选用；一题可被多个问卷使用
2	团队协作只能复制粘贴	可把题目共享给其他用户；对方能在自己的问卷里使用；共享后仍可用同一题
3	改了一个题，以前问卷里的题也跟着变了，数据对不上	已发布问卷的题目不能被修改；要改就生成新题目；旧问卷保持原内容
4	改完想找回旧版本，也想知道新题从哪改来	可看历史版本；可知道来源；可恢复旧版本（v1→v2→v3 关系可见）
5	同一题想稍微改一下用在不同问卷，又不想影响原题	可基于一题创建新版本；新版本可单独修改；不同问卷可用不同版本
6	想改题但不知道哪些问卷在用它	可查看题目被哪些问卷使用；修改时自己决定是否出新版本
7	希望把常用题整理成题库	可管理题库、把题加入题库、从题库选题、题库里的题可更新版本
8	统计只能看某个问卷，想看一道题在**所有**问卷中的回答情况	单题跨问卷统计（如"你的年龄"在所有问卷中的总体分布）

任务约束：教师明确要求第二阶段必须基于第一阶段现有代码修改，不得推倒重做；报告需记录"原设计 → 新需求问题 → 为什么这样改 → 改了哪里 → 数据结构变化 → 兼容性"。

二、原设计回顾与影响分析

第一阶段有两条设计决策与本次需求直接相关，其中一条意外地成了第二阶段的地基：

1. **题目是独立集合，不内嵌在问卷里。**`questions` 集合每道题一个文档，问卷只存 `question_count` 冗余计数。当时这样拆是为了编辑期并发安全，但它让第二阶段的"题目复用"有了天然的挂载点——题目本来就不是一个问卷文档的附属品。
2. **发布后冻结题目结构。**`draft → published → closed` 单向状态机，所有题目增删改接口入口处都有 `assertDraft` 守卫。需求 3（已发布问卷题目不可修改）在第一阶段**已经由这条规则完整满足**，本次不需要新增代码，只需要在新的题库场景下回归验证（用例 T82）。

真正的新问题是：第一阶段里题目从属于某一个问卷（`survey_id` 必填），不存在"一道题属于多个问卷"的表示方式；也没有"同一道题的多个版本"的概念。这两点必须引入新的数据结构，而引入新结构时最大的风险恰恰是需求 3——如果处理不当，题库的"更新"会反过来破坏问卷快照。

三、设计方案变更：三个核心决策

3.1 决策一：问卷侧存"快照"，不是"引用"

备选方案：问卷题只存 `bank_question_id`，读取时 join 题库最新版本。

否决理由：这正是需求 3 要禁止的行为。题库一更新，所有已发布问卷的题目内容随之变化，历史答卷的题干、选项立即失去解释依据——数据对不上，问题从根上出在"问卷题与题库题共享同一份可变状态"。

最终方案：从题库选题时，把题目内容（题干、选项、约束）**整体拷贝**为

问卷自己的题目文档；`bank_question_id` 与 `bank_version` 只作为溯源字段记录"来自题库哪个版本"。此后题库无论怎么出新版本，问卷快照一个字节都不变。需求 3（旧问卷不变）、需求 5（不同问卷用不同版本）由这一个决策同时满足，且不依赖任何额外状态同步。

3.2 决策二：题库题"版本即文档"

修改题库题**永远不原地覆盖**，而是插入一个新文档：`version + 1`，`parent_id` 指向修改前的版本，同一棵版本树的文档共享 `root_id`（v1 的根是它自己）。

对比被否决的"数组内嵌 history"方案：内嵌数组随修改次数无限增长；两个并发修改会在数组层面互相覆盖；更重要的是数组里的历史版本无法被"分别引用"——而需求 5 要求不同问卷分别使用 v1 和 v2，只有独立文档才能做到。

这个结构下，需求 4 的三个动作都有自然实现：看历史 = 按 `{root_id, version}` 列出版本链（`parent_id` 串出 v1→v2→v3 的关系）；知道来源 = 读 `parent_id`；恢复旧版本 = 以旧版本内容在树末端追加一个新版本（v3 内容等于 v1），历史一条不删。

3.3 决策三：共享是"树级"语义

`shared_with`（用户 id 数组）内嵌在题库题文档上，共享 / 取消共享对整棵版本树 `updateMany`。有两个容易漏掉的细节：

- 作者共享 v1 之后又改出 v2，若 v2 不继承 `shared_with`，被共享者会"丢题"——所以新版本文档创建时显式继承来源版本的共享列表；
- 被共享者的权限是"读 + 选题进问卷 + fork"，**不能**修改原题（403）。团队场景由 `fork` 收尾：fork 生成一棵 owner 是自己的新树，之后各改各

的，从根上避免写写冲突。

四、数据结构变化与兼容性

4.1 新增集合 bank_questions

字段	类型	说明
owner_id	ObjectId → users	创建者；决定修改/共享/恢复权限
type	String 枚举	七种题型，与 questions 一致
title / required / options / constraints	同 questions	题目内容本体
version	Number ≥ 1	树内版本号，从 1 开始
root_id	ObjectId → bank_questions	版本树根（v1 的 _id；v1 自身 root_id = 自身 id）
parent_id	ObjectId / null	修改或 fork 的来源版本；null 表示原创 v1
shared_with	[ObjectId]	被共享用户列表（树级语义）
created_at / updated_at	Date	时间戳

4.2 questions 集合新增字段

字段	类型	说明
bank_question_id	ObjectId / null	溯源：来自题库哪个文档；非题库题为 null
bank_version	Number / null	选题时的版本号快照

题目内容本体不变——快照在加入问卷那一刻已完整拷贝。

4.3 新增索引

索引	支撑的查询

索引	支撑的查询
{owner_id:1, updated_at:-1}	我的题库列表
{root_id:1, version:-1}	版本树查询、取每棵树最新版本
{shared_with:1}	"共享给我的"反查
questions.{bank_question_id:1}	使用情况反查、跨问卷统计的引用定位

4.4 兼容性

- **旧数据零迁移**：bank_question_id / bank_version 均可空，第一阶段创建的所有问卷题两个字段为 null，读取端按"非题库题"分支处理。第一阶段问卷、答卷、统计全部原样可用。
- **答卷零影响**：responses.answers 通过 question_id（问卷快照的 id）关联，与题库完全解耦；题库演进不会触碰任何答卷。
- **接口向后兼容**：POST /surveys/:id/questions 原有请求体（type/title/options…）行为不变；bank_question_id 是新增的可选字段。

五、接口设计与实现要点

新增 routes/bank.js，挂载于 /api/bank-questions，共 11 个端点覆盖八项需求：

需求	端点
创建 / 列表 / 详情	POST /、GET /?scope=mine|shared、GET /:id
修改（= 新版本）	PUT /:id（部分更新：未传字段继承当前版本）
多版本并存 / 接收共享	POST /:id/fork
版本历史 / 恢复旧版本	GET /:id/versions、POST /:id/revert
共享 / 取消共享	POST /:id/share、POST /:id/unshare

需求	端点
使用情况	`GET /:id/usage`
跨问卷统计	`GET /:id/stats`
问卷题存为常用题	`POST /from-question`

实现要点：

- **权限三级模型**：owner（可改/共享/恢复）、被共享者（可读/选题/fork）、其他（404 级隔离，实际上返回 403 防止探测）。所有端点入口统一走 `loadAccessibleBankQuestion()`。
- **列表去版本噪音**：题库列表按 `{root_id, version:-1}` 排序后内存分组取每组第一条，每棵树只展示最新版本，避免 v1/v2/v3 铺满一屏。
- **跨问卷统计是一条聚合管道**：先按溯源字段定位该题所有版本在所有问卷中的快照，然后 `$match(survey_id)` → `$unwind(answers)` → `$match(question_id)`，再按题型分支——选项类再 `$unwind option_keys` 后 `$group` 计数；数字类 `$min/$max/$avg`；文本类 `$push` 后 `$slice` 最近 20 条；日期类按值 `$group`。计算全部下沉数据库侧。
- **开发中修掉的两个自身设计错误**：① `root_id` 一度设为 `required`，但 v1 需要先插入拿到 `_id` 才能回填 root_id，`create()` 在插入瞬间就跑校验直接 400——改为去掉 required、由写路径约定保证；② 修改接口初稿是全量语义，只传题干会把选项清空——改为部分更新（未传字段继承当前版本）。

六、前端页面

题库入口加入全站导航（我的问卷 / 题目库 / 退出），像素游戏风与第一阶段完全一致（复用既有类名，零新增 CSS）：



题库管理页：版本标签、修改/版本历史/共享/复制为新题/使用情况/跨问卷统计



问卷编辑页：新增"从题库选题"按钮（草稿状态），题目卡片显示"题库题 v1"溯源标签

题库页支持：我的题目 / 共享给我的两个页签；新建题目；每题的"修改（新版本）"、版本历史弹窗（版本链 + 一键恢复旧版本）、共享弹窗（按用户

名共享/取消）、使用情况弹窗（哪些问卷、用的哪个版本）、跨问卷统计弹窗（选项分布条形图、数字 min/max/avg、文本样本、日期分布）。

编辑页：草稿状态显示"从题库选题"按钮，弹窗内可切换"我的题目 / 共享给我的"页签并一键选用；每道题卡片有"存为常用题"按钮，把问卷题保存进题库；已带溯源的题显示"题库题 vN"标签。

七、测试

新增 `tests/api.bank.test.js`，T70～T88 共 19 条用例，每条对应一项需求或一条边界：

用例组	覆盖内容
T70 – T72	创建题库题（version=1、root_id=自身）、我的列表、他人隔离与未登录 401
T73 – T75	修改生成新版本（v2，parent 指向 v1）、版本链可查、revert 恢复 v1 生成 v3 且历史完整
T76 – T78	共享后对方可见、Bob 不能改 Alice 的题（403）但可 fork 后自由修改、越权 403、共享给不存在用户 404
T79 – T80	问卷题存入题库 + 从题库选题（快照 + 溯源字段）；**需求③核心回归：题库出 v2 后问卷快照仍为 v1 内容**
T81 – T83	使用情况反查、已发布问卷不能再从题库加题（409）、取消共享后不可见
T84 – T88	跨问卷统计完整数据流：两卷引用同一题各填答卷后，选项计数跨卷累计（2/1/1）、usage 反查、数字题 min/max/avg、未使用题返回全零

全量测试结果：**88 条用例全部通过**（第一阶段 69 条回归无破坏 + 第二阶段 19 条新增）。

八、AI 使用记录

本阶段继续使用对话式 AI 编码助手，日志 #21～#25（完整表格见 `docs/ai-usage-log.md`）。关键记录：

#	用途	是否修改	说明
21	八条需求的分析与方案取舍	修改后采用	AI 初稿建议"问卷引用题库并跟随最新版"，与需求 3 冲突，改为快照 + 溯源
22	题库模型与路由实现	修改后采用	修掉 root_id required 400、全量更新清空选项两处错误
23	前端题库页与选题弹窗	直接采用	复用像素风类名
24	19 条测试	修改后采用	修掉二次连库挂住、漏 teardown 两处；补上 T80 关键回归
25	跨问卷统计管道	修改后采用	文本样本加 $slice 限量

真实对话节选（完整版见 `docs/ai-usage-log.md` §5.3）：

- **我**：八条需求里最要紧的是"改题不能影响已做好的问卷"。题库题是不是让问卷直接引用，然后跟着最新版走？
- **AI**：可以让问卷题存 `bank_question_id` 引用题库文档，读取时 join 最新版本。这样选题复用最直接。
- **我**：不行，这正好违反第三条——老师改了题，去年发布的问卷内容就变了，收集的数据全对不上。
- **AI**：那就反过来：选题时把题目内容整体拷贝进问卷（快照），`bank_question_id` 只记来源用于反查和统计。题库题修改永远生成新版本不覆盖。这样第三条自动满足，第五条"不同问卷用不同版本"也自动成立。

- **我**：创建题库题一直 400，`details: ["root_id"]`，可我明明在 save 前回填了。
- **AI**：`Model.create()` 在插入那一刻就跑 schema 校验，此时 `root_id` 还是 null（required 校验失败），根本轮不到你后面的回填。把 `required: true` 去掉，回填改由写路径约定保证。

九、总结

这次需求变更是一次真正的"演进"练习：第一阶段所有改动都建立在既有代码之上——题目独立集合让题库有了挂载点，发布冻结规则让需求 3 天然满足，统一错误结构与测试基建让 19 条新用例第一天就能跑进既有套件。八项需求最终收敛为两个数据结构决策（快照溯源、版本即文档）和一个权限模型（树级共享），没有推翻任何一张已有的表，旧数据零迁移。

如果给"设计为变化留余地"找一个具体例证：第一阶段注释里写的"`schema_version` 是给第二阶段预留的"和"第二阶段要按题型扩展时只需新文档多几个字段"，在这次迭代中分别以"无需启用"和"新增 bank_questions 集合"的方式兑现——前者因为快照方案比预想的更彻底，后者因为题库本质上是"不属于任何问卷的题目"，恰好落回第一阶段的拆分决策上。